

· [KLDP.org](#) · [KLDP Wiki](#) · [KLDP.net](#) · [통합 검색](#) ·



[Subversion Book/Reference](#)

[🔍](#) [📄](#) [🔗](#) [🔍](#) [📄](#) [🔗](#) [XML](#)

[FrontPage](#) | [FindPage](#) | [TitleIndex](#) | [RecentChanges](#) |

[SubversionBook/BasicConcepts](#) › [SubversionBook/BranchingAndMerging](#) › [SubversionBook/RepositoryAdministration](#) › [SubversionBook/AdvancedTopics](#) › [SubversionBook/DeveloperInformation](#) › [SubversionBook/Reference](#)

# 1Chapter. Subversion 레퍼런스

## Table of Contents

### 1.1. Subversion 커맨드 라인 클라이언트: [svn](#)

#### 1.1.1. [svn](#)의 스위치

#### 1.1.2. [svn](#) 서버 커맨드

[svn add](#) -- 파일이나 디렉토리를 추가합니다.  
[svn cat](#) -- 지정한 파일 또는 URL의 내용을 표시합니다.  
[svn checkout](#) -- 저장소(repository)로부터 작업 카피를 체크아웃 합니다.  
[svn cleanup](#) -- 작업 카피를 재귀적으로 정상화한다.  
[svn commit](#) -- 작업 카피의 변경점을 저장소(repository)에 보냅니다.  
[svn copy](#) -- 작업 카피나 저장소(repository)중의, 파일이나 디렉토리를 카피.  
[svn delete](#) -- 작업 카피나 저장소(repository)로부터 파일 또는 디렉토리를 삭제합니다.  
[svn diff](#) -- 두 개의 패스동안의 차이점을 표시합니다.  
[svn export](#) -- 디렉토리 트리의 export  
[svn help](#) -- 헬프  
[svn import](#) -- PATH를 URL에 대해서 재귀적으로 커밋합니다.  
[svn info](#) -- PATH에 대한 정보를 표시합니다.  
[svn list](#) -- 저장소(repository)중의 디렉토리 엔트리를 일람표 가리킵니다.  
[svn log](#) -- 커밋 로그 메시지의 표시.  
[svn merge](#) -- 두 소스의 차이를 작업 카피 패스에 반영합니다.  
[svn mkdir](#) -- 버전 관리하에 있는 새로운 디렉토리를 만듭니다.  
[svn move](#) -- 파일이나 디렉토리를 이동한다.  
[svn propdel](#) -- 아이টে으로부터 속성을 삭제합니다.  
[svn propedit](#) -- 버전 관리되고 있는 하나 이상의 아이템의 속성을 편집한다.  
[svn propget](#) -- 속성의 값을 표시합니다.  
[svn proplist](#) -- 모든 속성을 일람표 가리킵니다.  
[svn propset](#) -- 파일, 디렉토리, 리비전의 PROPPNAME의 값(을)을 PROPVAL로 설정한다.  
[svn resolved](#) -- 작업 카피의 파일 또는 디렉토리의 "충돌" 상태를 잠기 들여다 봅니다.  
[svn revert](#) -- 로컬 파일에의 모든 편집을 취소합니다.  
[svn status](#) -- 작업 카피에 있는 파일이나 디렉토리 상태를 표시.  
[svn switch](#) -- 작업 카피를 다른 URL에 갱신합니다.  
[svn update](#) -- 작업 카피의 갱신.

### 1.2. [svnadmin](#)

#### 1.2.1. [svnadmin](#) 스위치

#### 1.2.2. [svnadmin](#) 서버 커맨드

[svnadmin list-unused-dblogs](#) -- 어느 로그 파일이 안전하게 삭제 가능한가를 Berkeley DB에 문의.  
[svnadmin create](#) -- REPOS\_PATH에 신규의 빈 저장소(repository)를 만듭니다.  
[svnadmin dump](#) -- 파일 시스템의 내용을 표준 출력에 덤프 합니다.  
[svnadmin help](#) --  
[svnadmin load](#) -- 표준 입력으로부터 "덤프 형식"의 데이터를 읽어들이니다.  
[svnadmin lstxns](#) -- 커밋되어 있지 않은 모든 트랜잭션(transaction)의 이름의 표시.  
[svnadmin recover](#) -- 저장소(repository)중의 없어진 상황의 복원.  
[svnadmin rmtxns](#) -- 저장소(repository)로부터 트랜잭션(transaction)을 삭제합니다.  
[svnadmin setlog](#) -- 리비전에 로그 메시지를 설정합니다.

### 1.3. [svnlook](#)

#### 1.3.1. [svnlook](#) 스위치

#### 1.3.2. [svnlook](#) 서버 커맨드

[svnlook author](#) -- 처리한 사람의 표시.  
[svnlook cat](#) -- 파일의 내용을 표시합니다.  
[svnlook changed](#) -- 변경된 패스를 표시합니다.  
[svnlook diff](#) -- 일자를 표시합니다.  
[svnlook diff](#) -- 변경된 파일, 디렉토리의 차분을 표시합니다.  
[svnlook dirs-changed](#) -- 변경이 있던 디렉토리를 표시합니다.  
[svnlook help](#) --  
[svnlook history](#) -- 저장소(repository)중의 패스의 히스토리에 관한 정보를 표시합니다 (혹은 패스가 지정되지 않았던 경우에는 루트 디렉토리가 됩니다).  
[svnlook info](#) -- 작업자, 일자, 로그 메시지의 크기, 로그 메시지를 표시합니다.  
[svnlook log](#) -- 로그 메시지를 표시합니다.  
[svnlook proplist](#) -- 버전화 된 파일과 디렉토리의 속성의 이름과 값을 표시합니다.  
[svnlook tree](#) -- 트리를 표시합니다.  
[svnlook youngest](#) -- 제일 최근의 리비전 번호를 표시합니다.

### 1.1.

이 장은 Subversion의 완전한 레퍼런스입니다. 커맨드 라인 클라이언트 ([svn](#))(와)과 그 모든 서버 커맨드, 한층 더 저장소(repository) 관리 프로그램 ([svnadmin](#)와[svnlook](#))(와)과 그 모든 서버 커맨드입니다.

## 1.1. Subversion 커맨드 라인 클라이언트: [svn](#)

커맨드 라인 클라이언트를 사용하려면 [svn](#) (이)라고 입력하고 나서, 사용하고 싶은 서버 커맨드를 입력해 주세요. [\[1\]](#), 어떤 스위치나 대상이 된다 타겟으로 붙어도 그렇습니다 서버 커맨드와 스위치의 맞은 특징의 순서는 없습니다. 예를 들어 [svn status](#): (을)를 사용하는 경우에, 이하는 모두 유효합니다:

Login:

Password:

[Join](#)

The attacker must vanquish;  
the defender need only  
survive.

[LinuxdocSgml/NIS.debian-KLDP](#)  
[Tomcat-HOWTO](#)  
[윈도우즈한글글꼴](#)  
[InterviewEngineer](#)



kss@kldp.org

```
$ svn -v status
$ svn status -v
$ svn status -v myfile
```

클라이언트 커멘드의 이용법에 대해서는, 좀 더 많은 예가 >에 있어, 속성의 관리에 대해서는 >에 있습니다.

### 1.1.1. svn 의 스위치

Subversion 는 서브 커멘드 마다 다른 스위치를 가져 있습니다만, 모두 글로벌하게 일합니다 즉 각각의 스위치는 함께 이용되는 서브 커멘드에 관계없이 같은 의미 (을)를 가집니다. 예를 들어 `--verbose (-v)` (은)는 항상, 어느 서브 커멘드와 함께 사용해도 "장황한 출력"을 의미합니다.

`--diff-cmd CMD`

파일간의 차이를 표시하기 위해서 사용하는 외부 프로그램을 지정합니다. **svn diff**가 기동되면(자), 디폴트에서는 unidiff diff 형식인 Subversion의 내부 diff 엔진이 이용됩니다. 외부 diff 프로그램 (을)를 이용하고 싶은 경우는, `--diff-cmd`를 사용해 주세요. `--extensions`스위치로 diff 프로그램을 지정하는 일도로 옵니다. (자세하게는 이 장의 나중에 설명하겠습니다).

`--diff3-cmd CMD`

파일을 merge 하기 위해서 사용하는 외부 프로그램을 지정합니다.

`--dry-run`

커멘드의 실행을 그대로 더듬습니다만, 실제의 동작은 하지 않습니다 작업 카피에 대해서도, 저장소(repository)에 대한 커멘드에도 사용할 수 있습니다.

`--editor-cmd CMD`

로그 메시지가 속성치를 편집하는데 사용하는 외부 프로그램을 지정합니다.

`--encoding ENC`

Subversion 에 커밋의 메시지를 준비한 charset로 encode 하도록(듯이) 지시합니다. 디폴트는 당신의 operating system의 원래의 locale 로, 커밋 메시지가 encode의 경우에는 그것을 지정할 필요가 있습니다.

`--extensions(-x) "ARGS"`

Subversion 가 파일간의 차이를 얻을 때에 사용하는 외부 diff 커멘드에 건네주지 않고 (이)라고는 안 되는 하나 이상의 인수를 지정합니다. 복수의 인수를 건네주고 싶다면 경우는, 그것들 모든 것을 인용부호로 묶지 않으면 안됩니다. (예를 들어, **svn diff --diff-cmd /usr/bin/diff -x "-b -E"**). 이 스위치는 `--diff-cmd`스위치도 지정했을 경우 **에 마셔** 이용할 수가 있습니다.

`--file (-F) FILENAME`

지정한 서브 커멘드에 대해, 이 스위치의 인수로서 건네준 파일의 내용이 사용됩니다.

`--force`

특정의 커멘드 또는 조작의 실행을 강제합니다. Subversion 가 통상이라면 거부하는 것 같은 몇개의 조작이 있습니다만, 이 강제 스위치를 사용해, Subversion 에 "그것을 하먼(자) 어떻게 되는 일이 있을까는 알고 있기 때문에, 어쨌든 그것을 시켜 줘"라고 전할 수가 있습니다. 이 스위치는, 전자 공작이나 무엇인가로, 마지막에 전원을 넣는 상황에 답습니다무엇을 하려 하고 있는지 잘 이해하고 있지 않으면 반드시 싫은 눈을 당하겠지요.

`--force-log`

`--messages (-m)`나 `--file (-F)` 옵션에 건네주는 의심스러운 인수를 올바른 것으로서 강제적으로 받아들인다 같게 지시합니다. 디폴트에서는, Subversion 는 올바른 서브 커멘드로 어느 해석하는 대신에, 그러한 인수가 실수이라고 하여 에러를 생성합니다. 예를 들어, 버전화 된 파일의 패스를 `--file (-F)` 옵션에 건네주면(자), Subversion 는 그 패스가 조작 대상으로 해 의도되고 있었다고 생각하지 않고, 그 인수에는 실수가 있으면(자) 간주, 로그 메시지를 위한 버전화되어 있지 않은 원파일을 준비하는 것에 실패해 버립니다. 자신의 의도를 선언해, 이런 종류의 에러 (을)를 무시하기 위해서는, `--force-log`옵션을 로그 메시지 (을)를 받아들이는 커멘드에 대해서 지정할 필요가 있습니다.

`--help (-h or -?)`

하나 이상의 서브 커멘드와 함께 사용해, 각각의 서브 커멘드의 편입 헬프 텍스트를 표시합니다. 이 스위치만을 단독으로 사용하면 (자), 일반적인 커멘드 클라이언트의 헬프 텍스트를 표시합니다.

`--notice-ancestry`

차분을 계산할 경우에, 히스토리에 주의합니다.

`--incremental`

(다른 출력과) 연결하는데 적합한 형식에서 출력을 표시합니다.

`--message (-m) MESSAGE`

커멘드 라인상에서 커밋 메시지를 지정합니다. 이 스위치의 나중에, 예를 들어:

```
$ svn commit -m "They don't make Sunday. "
```

`--new ARG`

ARG 를 새로운 타겟으로서 이용합니다.

`--no-auth-cache`

인증 정보(유저명이나 패스워드등)를 Subversion 관리 디렉토리에 캐쉬하지 않게 지시합니다.

`--no-diff-deleted`

삭제된 파일의 차분을 표시하지 않게 Subversion에 지시합니다. 디폴트에서는 파일을 삭제했을 때의 **svn diff**의 출력은, 파일은 삭제되지 않고 제로 아르바이트로 남아 있는 것과 같은 형태가 됩니다.

**--no-ignore**

svn:ignore 속성에 성났 했기 때문에 통상이라면 생략 되는 것 같은 파일의 스테이투스 일람을 표시합니다. > 에 한층 더 자세한 정보가 있습니다.

**--non-interactive**

인증이 실패하거나 충분한 허가가 없을 때에, 인증 요구하는 것을 억제합니다. (예를 들어, 유저명과 패스워드) 이것은, Subversion 을 자동 스크립트중에서 실행하고 있는 것 같은 때로 인증 요구시키는 것보다도 단지 실패하는 편이 편리한 경우에 도움이 됩니다.

**--non-recursive (-N)**

서브 디렉토리에 대해서 서브 커멘드를 재귀적으로 실행하는 것을 억제합니다. 대부분의 서브 커멘드는 디폴트로 재귀적으로 실행됩니다만, 몇개의 서브 커멘드보통은, 작업 카피의 변경에 대한 삭제나 취소가 일어나는 것 같은 것은 그런 것은 없습니다.

**--old ARG**

ARG 를 낡은 타겟으로서 이용합니다.

**--password/PASS**

커멘드 라인상에서 인증용 패스워드를 지정합니다이것을 지정하지 않으면, 필요한 장소에서 Subversion 는 패스워드 입력을 요구해 옵니다.

**--quiet (-q)**

실행중에 중요한 정보만을 표시하도록(듯이) 지시합니다.

**--recursive (-R)**

서브 커멘드를 서브 디렉토리에 대해서 재귀적으로 실행한다 같게 합니다. 대부분의 서브 커멘드는 디폴트로 재귀적인 동작을 합니다.

**--relocate FROM TO [PATH...]**

**svn switch** 서브 커멘드와 함께 사용하는 것으로 작업 카피 하지만 참조하고 있는 저장소(repository)의 장소를 변경합니다. 이것은, 저장소(repository)의 장소가 이동해도, 벌써 존재하고 있는 작업 카피를 계속해 이용하고 싶은 경우 에 편리합니다. 예로서 **svn switch**를 참조해 주세요.

**--revision (-r) REV**

특정의 조각으로, 리비전(또는 리비전의 범위)을 설정합니다. 리비전은 리비전 스위치의 인수로서 번호, 키워드, 일자(이 경우는 안의모를 사용해)의 어떤 것인가로 지정할 수가 있습니다. 리비전을 돌지정할 때에는 코론으로 단락짓습니다. 예를 들어:

```
$ svn log -r 1729
$ svn log -r 1729:HEAD
$ svn log -r 1729:1744
$ svn log -r {12/04/01}:{2/17/02}
$ svn log -r 1729:{2/17/02}
```

자세하게는 > 을 봐 주세요.

**--revprop**

파일이나 디렉토리의 속성의 대신에, 리비전의 속성에 대해 조작하도록(듯이) 합니다. 이 스위치를 사용하는 경우는 **--revision(-r)**를 사용해 리비전도 건네줄 필요가 있습니다. 버전화하지 않는 속성에 대한 자세한 것은 > (을)를 봐 주세요.

**--show-updates (-u)**

작업 카피의 어느 파일이 최신일가의 정보를 표시합니다. 이것은 실제로 자신에게 작업 파일을 갱신하지 않습니다만약 **svn update**를 실행했다고 하면(자), 어느 파일이 갱신되는지를 표시할 뿐입니다.

**--strict**

Subversion은 엄밀한 의미를 사용하게 됩니다. 자세하게는 > (을)를 참조해 주세요.

**--targets FILENAME**

Subversion 에 커멘드 라인상의 모든 파일을 일람표시 하는 대신에 지정한 파일의 일람을 취득하도록 지시합니다.

**--username/NAME**

커멘드 라인상에서의 인증에, 지정한 유저 명칭을 사용하도록(듯이) 지시합니다그렇지 않으면, 필요한 장소에서, Subversion은 그 입력을 유저에게 요구합니다.

**--verbose (-v)**

서브 커멘드 실행시에, 할 수 있는 사나워지고 많은 정보를 표시하도록(듯이) 지시합니다. Subversion 는, 추가 필드, 모든 파일의 상세 정보, 동작 내용에 대한 추가 정보등을 표시하게 됩니다.

**--version**

클라이언트 프로그램의 버전 번호를 표시합니다. 이 정보는 버전 번호외, Subversion 저장소(repository)에 클라이언트 프로그램 하지만 액세스 하기 위해서 이용할 수 있는 모듈의 일람도 표시합니다.

**--xml**

XML 포맷으로 출력합니다.

### 1.1.2. svn 서브 커맨드

#### Table of Contents

[svn add](#) -- 파일이나 디렉토리를 추가합니다.  
[svn cat](#) -- 지정한 파일 또는 URL의 내용을 표시합니다.  
[svn checkout](#) -- 저장소(repository)로부터 작업 카피를 체크아웃 합니다.  
[svn cleanup](#) -- 작업 카피를 재귀적으로 정상화한다.  
[svn commit](#) -- 작업 카피의 변경점을 저장소(repository)에 보냅니다.  
[svn copy](#) -- 작업 카피나 저장소(repository)중의, 파일이나 디렉토리를 카피.  
[svn delete](#) -- 작업 카피나 저장소(repository)로부터 파일 또는 디렉토리를 삭제합니다.  
[svn diff](#) -- 두 개의 패스동안의 차이점을 표시합니다.  
[svn export](#) -- 디렉토리 트리의 export  
[svn help](#) -- 헬프  
[svn import](#) -- PATH를 URL에 대해서 재귀적으로 커밋합니다.  
[svn info](#) -- PATH에 대한 정보를 표시합니다.  
[svn list](#) -- 저장소(repository)중의 디렉토리 엔트리를 일람표 가리킵니다.  
[svn log](#) -- 커밋 로그 메시지의 표시.  
[svn merge](#) -- 두 소스의 차이를 작업 카피 패스에 반영합니다.  
[svn mkdir](#) -- 버전 관리하에 있는 새로운 디렉토리를 만듭니다.  
[svn move](#) -- 파일이나 디렉토리를 이동한다.  
[svn propdel](#) -- 아이টে으로부터 속성을 삭제합니다.  
[svn propedit](#) -- 버전 관리되고 있는 하나 이상의 아이템의 속성을 편집한다.  
[svn propget](#) -- 속성의 값을 표시합니다.  
[svn proplist](#) -- 모든 속성을 일람표 가리킵니다.  
[svn propset](#) -- 파일, 디렉토리, 리비전의 PROPNAME 의 값 (을)를 PROPVAL로 설정한다.  
[svn resolved](#) -- 작업 카피의 파일 또는 디렉토리의 "충돌" 상태를 잡기 들여다 봅니다.  
[svn revert](#) -- 로컬 파일에의 모든 편집을 취소합니다.  
[svn status](#) -- 작업 카피에 있는 파일이나 디렉토리 상태를 표시.  
[svn switch](#) -- 작업 카피를 다른 URL에 갱신합니다.  
[svn update](#) -- 작업 카피의 갱신.

## svn add

### Name

svn add -- 파일이나 디렉토리를 추가합니다.

### 용법

svn add PATH...

### 설명

파일이나 디렉토리를 작업 카피에 추가해 저장소(repository)에 추가할 것을 예약 합니다. 실제의 추가는 커밋 명령으로 저장소(repository)에 업로드하여 이루어집니다. 커밋 전에 마음이 바뀌었을 경우엔 **svn revert**로 추가를 취소할 수 있습니다.

### 별명

없음

### 변경 대상

작업 카피

### 스위치

--targets FILENAME  
 --non-recursive (-N)  
 --quiet (-q)

### 예

작업 카피에 파일을 추가:

```
$ svn add foo.c
A      foo.c
```

디렉토리를 추가할 때 **svn add** 의 디폴트는 재귀적입니다:

```
$ svn add testdir
A      testdir
A      testdir/a
A      testdir/b
A      testdir/c
A      testdir/d
```

내부에 있는 파일을 추가하는 것 없이, 디렉토리만 추가한다 일이 생깁니다:

```
$ svn add --non-recursive otherdir
A      otherdir
```

## svn cat

### Name

svn cat -- 지정한 파일 또는 URL의 내용을 표시합니다.

### 용법

```
svn cat TARGET...
```

### 설명

지정한 파일 또는 URL의 내용을 표시합니다. 디렉토리의 내용의 표시에 대해서는, **svn list** 를 봐 주세요.

### 별명

없음

### 변경 대상

없음

### 저장소(repository)에의 액세스

발생

### 스위치

```
--revision (-r) REV
--username USER
--password PASS
```

### 예

체크아웃 하는 것 없이 저장소(repository)중의 readme.txt를 표시하고 싶다 경우:

```
$ svn cat http://svn.red-bean.com/repos/test/readme.txt
This is a README file.
You should read this.
```

**Tip:** 작업 카피가 최신은 아니다(인가, 작업 카피에 무엇인가 수정을 더했다 경우) 상태로, 작업 카피가 있는 파일의 HEAD 리비전을 보고 싶은 경우, 패스를 지정하면 **svn cat** 는 자동적으로 HEAD 리비전 에 액세스 합니다:

```
$ cat foo.c
This file is in my local working copy
and has changes that I've made.

$ svn cat foo.c
Latest revision fresh from the repository!
```

## svn checkout

### Name

svn checkout -- 저장소(repository)로부터 작업 카피를 체크아웃 합니다.

### 용법

```
svn checkout URL... [PATH]
```

### 설명

저장소(repository)로부터 작업 카피를 체크아웃 합니다. *PATH*가 생략 되면 카피처로서 URL의 베이스명이 사용됩니다. 복수의 URL가 지정되었을 경우에는, 각각이, PATH의 서브 디렉토리에 체크아웃 됩니다만 여기서의 서브 디렉토리의 이름은 URL의 베이스명이 됩니다.

## 별명

co

## 변경 대상

작업 카피가 새롭게 만들어집니다.

## 저장소(repository)에의 액세스

발생

## 스윗치

```
--revision (-r) REV
--quiet (-q)
--non-recursive (-N)
--username USER
--password PASS
--no-auth-cache
--non-interactive
```

## 예

'mine'로 불리는 디렉토리에 작업 카피를 체크아웃:

```
$ svn checkout file:///tmp/repos/test mine
A mine/a
A mine/b
Checked out revision 2.
$ ls
mine
```

두 다른 디렉토리를 두 개개의 작업 카피에 체크아웃:

```
$ svn checkout file:///tmp/repos/test file:///tmp/repos/quiz
A test/a
A test/b
Checked out revision 2.
A quiz/l
A quiz/m
Checked out revision 2.
$ ls
quiz test
```

두 다른 디렉토리를 두 개개의 작업 카피에 체크아웃 하지만, 양쪽 모두 'working-copies'로 불린다 디렉토리에 만든다:

```
$ svn checkout file:///tmp/repos/test file:///tmp/repos/quiz working-copies
A working-copies/test/a
A working-copies/test/b
Checked out revision 2.
A working-copies/quiz/l
A working-copies/quiz/m
Checked out revision 2.
$ ls
working-copies
```

체크아웃을 중단한다(가마타는, 네트워크 접속의 불편 등에 의해 체크아웃이 중단될까 할) 경우, 한번 더 순수한 체크아웃을 실행해도, 불완전한 작업 카피를 갱신하는 것도 재개할 수 있습니다:

```
$ svn checkout file:///tmp/repos/test test
A test/a
A test/b
^C
svn: The operation was interrupted
svn: caught SIGINT

$ svn checkout file:///tmp/repos/test test
A test/c
A test/d
^C
svn: The operation was interrupted
```

```
svn: caught SIGINT

$ cd test
$ svn update
A   test/e
A   test/f
Updated to revision 3.
```

## svn cleanup

### Name

svn cleanup -- 작업 카피를 재귀적으로 정상화한다.

### 용법

```
svn cleanup [PATH...]
```

### 설명

작업 카피를 재귀적으로 정상화하기 위해(때문에), 조작이 미완료의 락을 삭제합니다. "작업 카피 락"에러가 발생했을 때에는, 이 커멘드를 실행해, 락을 해제해, 작업 카피를 정상적으로 되돌려 주세요. >(을)를 봐 주세요.

### 별명

없음

### 변경 대상

작업 카피

### 저장소(repository)에의 액세스

발생하지 않는다

### 스윗치:

없음

### 예

**svn cleanup** 는 아무것도 출력하지 않기 때문에, 그다지 예는 없습니다. 만약, PATH 를 지정하지 않으면, '.'를 대신에 사용합니다.

```
$ svn cleanup
$ svn cleanup /path/to/working-copy
```

## svn commit

### Name

svn commit -- 작업 카피의 변경점을 저장소(repository)에 보냅니다.

### 용법

```
svn commit [PATH...]
```

### 설명

작업 카피의 변경점을 저장소(repository)에 보냅니다. --file인가, --message 의 옵션을 지정하지 않으면 **svn** 는 에디터를 기동해, 유저에게 커밋 메시지를 작성시킵니다. >에 있다 editor-cmd 의 장을 봐 주세요.

**Tip:** 커밋 처리를 시작해 Subversion가 메시지 작성을 위한 에디터를 기동 한 다음에도 아직 변경을 커밋하지 않고 중단할 수가 있습니다. 커밋을 취소하고 싶다면 커밋 메시지를 보존하지 않고 단지 에디터를 종료 해 주세요. Subversion은 커밋을 중단하는지, 메시지가없이 계속 하는지, 메시지를 재편집할까를 (들) 물어 옵니다.

### 별명

ci

ci (short for "check in" 의 단축형으로, "co"의 단축이 아닙니다. "co"는"checkout"의 단축형입니다. )

## 변경 대상

작업 카피, 저장소(repository)

## 저장소(repository)에의 액세스

발생

## 스위치

```
--message (-m) TEXT
--file (-F) FILE
--quiet (-q)
--non-recursive (-N)
--targets FILENAME
--force-log
--username USER
--password PASS
--no-auth-cache
--non-interactive
--encoding ENC
```

## 예

커멘트 라인으로 지정한 커밋 메시지와 커런트 디렉토리 ("")(을)를 암묵의 타겟으로서 단순한 변경을 커밋합니다:

```
$ svn commit -m "added howto section. "
Sending          a
Transmitting file data .
Committed revision 3.
```

파일foo.c (커멘트 라인으로 명시적으로 지정)의 변경점을, 파일msg의 내용을 커밋 메시지와 해 커밋:

```
$ svn commit -F msg foo.c
Sending          foo.c
Transmitting file data .
Committed revision 5.
```

커밋 메시지로써--file로 지정한 파일이 버전 관리하에 있는 경우, --force-log 스위치를 사용할 필요가 있다:

```
$ svn commit --file file_under_vc.txt foo.c
svn: The log message file is under version control
svn: Log message file is a versioned file; use '--force-log' to override.

$ svn commit --force --file-log file_under_vc.txt foo.c
Sending          foo.c
Transmitting file data .
Committed revision 6.
```

삭제 예고 파일을 커밋:

```
svn commit -m "removed file 'c'. "
Deleting         c

Committed revision 7.
```

## svn copy

### Name

svn copy -- 작업 카피나 저장소(repository)중의, 파일이나 디렉토리를 카피.

### 용법

```
svn copy SRC DST
```

### 설명

작업 카피 또는 저장소(repository)중의 파일을 카피합니다. SRC 와 DST 는, 작업 카피(WC) 상의 패스로도, URL에서도 괜찮습니다:



WC - WC

추가용으로 파일을 카피해, 추가 예고합니다. (with history).

WC - URL

WC의 카피를 직접 URL에 커밋.

URL - WC

URL를 WC에 체크아웃 해, 추가 예고한다.

URL - URL

완전한 서버상만에서의 카피. 이것은 보통 브랜치(branch)나 태그에 이용됩니다.

| Warning                                                                                  |
|------------------------------------------------------------------------------------------|
| 파일은 하나의 저장소(repository)의 내부에서만 카피 가능합니다. Subversion은 저장소(repository)간 카피를 서포트하고 있지 않습니다. |

## 별명

cp

## 변경 대상

카피처가 URL 인 경우는 저장소(repository).

카피처가 작업 카피의 패스인 경우는 작업 카피.

## 저장소(repository)에의 액세스

카피된 또는 카피처가 저장소(repository)인지, 소스 리버전 번호를 참조할 필요가 있는 경우에는 발생.

## 스윗치

```
--message (-m) TEXT
--file (-F) FILE
--revision (-r) REV
--quiet (-q)
--username USER
--password PASS
--no-auth-cache
--non-interactive
--encoding ENC
```

## 예

작업 카피중의 파일 또는 디렉토리를 카피( 카피 예고만다음의 커밋까지 저장소(repository)에는 아무것도 일어나지 않는다):

```
$ svn copy foo.txt bar.txt
A      bar.txt
$ svn status
A +   bar.txt
```

저장소(repository)중의 URL 에 작업 카피중의 파일 또는 디렉토리 (을)를 카피(동시에 커밋되므로, 커밋 메시지를 지정할 필요 있어):

```
$ svn copy near.txt file:///tmp/repos/test/far-away.txt -m "Remote copy. "
Committed revision 8.
```

저장소(repository)의 파일 또는 디렉토리를 작업 카피에 카피(카피의 예고만다음의 커밋까지 저장소(repository)에는 아무것도 일어나지 않는다):

**Tip:** 이것은 저장소(repository)중이 죽은 파일을 restore시키는 추천하는 방법입니다

```
$ svn copy file:///tmp/repos/test/far-away near-here
A      near-here
```

그리고 마지막에 두 개의 URL의 사이에 카피하는 방법:

```
$ svn copy file:///tmp/repos/test/far-away file:///tmp/repos/test/over-there -m "remote copy. "
```

Committed revision 9.

**Tip:** 저장소(repository)중의 리비전에 태그 짓고 하는 제일 간단한 방법 그 리비전(보통은 HEAD)을 태그 짓고 되었다 디렉토리에, 단지 **svn copy** 한다.

```
$ svn copy file:///tmp/repos/test/trunk file:///tmp/repos/test/tags/0.6.32-prerelease -m "tag tree"
```

Committed revision 12.

그 태그를 잇는 것을 실패할 필요는 없습니다 언제나라도 낡은 리비전을 지정해 태그 붙이고 하는 것이 할 수 있습니다:

```
$ svn copy -r 11 file:///tmp/repos/test/trunk file:///tmp/repos/test/tags/0.6.32-prerelease -m "Forgot to tag at rev 11"
```

Committed revision 13.

## svn delete

### Name

svn delete -- 작업 카피나 저장소(repository)로부터 파일 또는 디렉토리를 삭제합니다.

### 용법

```
svn delete PATH...
```

```
svn delete URL...
```

### 설명

PATH로 지정된 파일 또는 디렉토리는 다음의 커밋 그리고 삭제하는 것을 예고합니다. 파일(와 아직 커밋하고 있지 않다 디렉토리)는 즉시 작업 카피로부터 삭제됩니다. 이 커멘트는 버전화되어 있지 않은가 수정되어 있지 않은 것에 대해서는 동작하지 않습니다. --force 스위치를 사용하면 이 동작을 변경할 수 있습니다.

URL로 지정된 파일 또는 디렉토리는 직접 커밋을 발행하는 형태로 삭제됩니다. 복수의 URL 는 단일의 트랜잭션(transaction)와 해 불분할에 커밋됩니다.

### 별명

del, remove, rm

### 변경 대상

파일에 대해서 실행했을 경우는 작업 카피. URL에 대해서 실행했을 경우는 저장소(repository)

### 저장소(repository)에의 액세스

URL에 대해서 조작했을 경우만 발생

### 스위치

```
--force
--force-log
--message (-m) TEXT
--file (-F) FILE
--quiet (-q)
--targets FILENAME
--username USER
--password PASS
--no-auth-cache
--non-interactive
--encoding ENC
```

### 예

svn 를 사용해 작업 카피로부터 파일을 삭제하면(자), 단지 삭제가 예고될 뿐입니다. 파일은 다음의 커밋 그리고 저장소(repository)로부터 삭제됩니다.

```
$ svn delete myfile
```

```
D      myfile

$ svn commit -m "Deleted file 'myfile'. "
Deleting      myfile
Transmitting file data .
Committed revision 14.
```

URL를 삭제합니다만, 그것은 즉시 일어나므로 로그 메시지를 지정할 필요가 있습니다:

```
$ svn delete -m "Deleting file 'yourfile'" file:///tmp/repos/test/yourfile

Committed revision 15.
```

이 예는 작업 카피에 수정이 있는 파일을 강제 삭제하는 방법입니다:

```
$ svn delete over-there
svn: Attempting restricted operation for modified resource
svn: Use --force to override this restriction
svn: 'over-there' has local modifications

$ svn delete --force over-there
D      over-there
```

## svn diff

### Name

svn diff -- 두 개의 패스동안의 차이점을 표시합니다.

### 용법

```
svn diff [-r N[:M]] [TARGET...]
```

```
svn diff URL1[@N] URL2[@M]
```

### 설명

두 개의 패스동안의 차이점을 표시합니다. 각각의 TARGET 는 작업 카피 패스나 URL입니다. 만약 TARGET를 지정하지 않으면 '.'가 사용됩니다.

TARGET 가 URL라면 리비전 N와 M는--revision를 사용해 지정됩니다.

TARGET가 작업 카피 패스라면,--revision 스위치의 의미는 이하와 같이 됩니다:

#### --revision N:M

서버는 TARGET@N 와 TARGET@M 를 비교합니다.

#### --revision N

클라이언트는 TARGET@N와 작업 카피를 비교합니다.

#### (no --revision)

클라이언트는 TARGET의 베이스 리비전과 작업 카피를 비교합니다.

다른 구문을 사용하면(자) 서버는 각각 리비전 N와 M에 있는 URL1와 URL2 (을)를 비교합니다. 만약 N나 M가 생략 되면 HEAD의 값이 사용됩니다.

### 별명

di

### 변경 대상

없음

### 저장소(repository)에의 액세스

차이점을 아는데 필요한 경우는 발생합니다. 다만 작업 카피의 BASE 리비전을 취득하는 경우는 제외합니다.

### 스위치

```
--revision (-r) REV
--extensions (-x) "ARGS"
```

```
--non-recursive (-N)
--diff-cmd CMD
--username USER
--password PASS
--no-auth-cache
--non-interactive
--no-diff-deleted
```

## 예

BASE 리비전과 작업 카피를 비교합니다. (**svn diff** 의 제일 자주 있는 사용법):

```
$ svn diff COMMITTERS
Index: COMMITTERS
=====
--- COMMITTERS (revision 4404)
+++ COMMITTERS (working copy)
```

작업 카피의 변경을 이전의 리비전과 비교하는 방법:

```
$ svn diff -r 3900 COMMITTERS
Index: COMMITTERS
=====
--- COMMITTERS (revision 3900)
+++ COMMITTERS (working copy)
```

리비전 3000으로 리비전 3500을 '@'구문을 사용해 비교:

```
$ svn diff http://svn.collab.net/repos/svn/trunk/COMMITTERS@3000 http://svn.collab.net/repos/svn/trunk/COMMITTERS@3500
Index: COMMITTERS
=====
--- COMMITTERS (revision 3000)
+++ COMMITTERS (revision 3500)
```

리비전 3000으로 리비전 3500을 범위 지정으로 비교( 이 경우는 하나의 URL만을 건네주면 된다):

```
$ svn diff -r 3000:3500 http://svn.collab.net/repos/svn/trunk/COMMITTERS
Index: COMMITTERS
=====
--- COMMITTERS (revision 3000)
+++ COMMITTERS (revision 3500)
```

작업 카피가 있는 경우, 긴 URL 지정하는 것 없이 차이점을 취득할 수가 있습니다:

```
$ svn diff -r 3000:3500 COMMITTERS
Index: COMMITTERS
=====
--- COMMITTERS (revision 3000)
+++ COMMITTERS (revision 3500)
```

--diff-cmd *CMD* -x 를 사용해 외부 diff 프로그램에 직접 인수를 건네줍니다

```
svn diff --diff-cmd /usr/bin/diff -x "-i -b" COMMITTERS
Index: COMMITTERS
=====
```

```
0a1, 2
This is a test
```

## svn export

### Name

svn export -- 디렉토리 트리의 export

### 용법

```
svn export [-r REV] URL [PATH]
```

```
svn export PATH1 PATH2
```

## 설명

최초의 구문에서는 지정된 URL의 저장소(repository)로부터 디렉토리 트리를 export 합니다. 이 차이, REV 가 지정되면 리비전 REV로부터, 그렇지 않으면 HEAD의 리비전이 이용되어 결과는 PATH에 출력됩니다. PATH가 생략 되면 URL의 마지막 부분이 로컬 디렉토리 명칭으로서 이용됩니다.

두번째의 구문에서는 PATH1로 지정된 로컬 작업 카피를 PATH2에 출력 합니다. 모든 작업 카피에의 변경은 보존됩니다만, 버전 관리 하에 없는 파일은 카피되지 않습니다.

## 별명

없음

## 변경 대상

로컬 디스크

## 저장소(repository)에의 액세스

URL로부터의 export의 경우만 발생

## 스위치

```
--revision (-r) REV
--quiet (-q)
--username USER
--password PASS
--no-auth-cache
--non-interactive
```

## 예

작업 카피로부터의 export( 모든 파일과 디렉토리를 하나 하나 표시하지 않는다):

```
$ svn export a-wc my-export
pantheon: /tmp
```

저장소(repository)로부터 직접 export( 모든 파일과 디렉토리를 표시):

```
$ svn export file:///tmp/repos my-export
A my-export/test
A my-export/quiz
Exported revision 15.
```

## svn help

### Name

```
svn help -- 헬프
```

### 용법

```
svn help [SUBCOMMAND...]
```

## 설명

Subversion을 사용하는 경우의 최선의 가이드로, 거기에 비교하면(자) 이 책 등은 미치지 않습니다!

## 별명

?, h

## 변경 대상

없음

## 저장소(repository)에의 액세스

발생하지 않는다

### 스윗치

```
--version
--quiet (-q)
```

## svn import

### Name

svn import -- PATH를 URL에 대해서 재귀적으로 커밋합니다.

### 용법

```
svn import [PATH] URL
```

### 설명

PATH의 카피를 재귀적으로URL 에 커밋합니다. PATH 가 없으면 '.'가 가정됩니다. 필요에 따라서 친디렉토리가 저장소(repository)에 만들어집니다.

### 별명

없음

### 변경 대상

저장소(repository)

## 저장소(repository)에의 액세스

발생한다

### 스윗치

```
--message (-m) TEXT
--file (-F) FILE
--quiet (-q)
--non-recursive (-N)
--username USER
--password PASS
--no-auth-cache
--non-interactive
--force-log
--encoding ENC
```

### 예

이것은, 로컬 디렉토리'myproj'를 저장소(repository)의 근원에 임포트 합니다:

```
$ svn import -m "New import" myproj http://svn.red-bean.com/repos/test
Adding          myproj/sample.txt

Transmitting file data .....
Committed revision 16.
```

이것은 로컬 디렉토리'myproj'를 저장소(repository)의'trunk/vendors' 에 임포트 합니다. 디렉토리'trunk/vendor'는 임포트 하기 전에 존재하고 있지 않아도 괜찮습니다 **svn import** (은)는 재귀적으로 디렉토리를 만듭니다:

```
$ svn import -m "New import" myproj W
http://svn.red-bean.com/repos/test/trunk/vendors/myproj
Adding          myproj/sample.txt

Transmitting file data .....
Committed revision 19.
```

## svn info

### Name

svn info -- PATH에 대한 정보를 표시합니다.

### 용법

```
svn info [PATH...]
```

### 설명

작업 카피의 패스에 대한 정보를 표시합니다. 이것에는:

- 패스
- 이름
- URL
- 리비전
- 노드의 종류
- 마지막에 수정한 사람
- 마지막에 수정한 리비전
- 마지막에 수정한 일시
- 마지막에 갱신한 텍스트
- 마지막에 갱신한 속성
- 체크 섬

### 별명

없음

### 변경 대상

없음

### 저장소(repository)에의 액세스

발생하지 않는다

### 스윗치

```
--targets FILENAME
--recursive (-R)
```

### 예

**svn info** 는 작업 카피중의 파일 또는 디렉토리에 관한 유용한 정보를 표시합니다. 다음은 파일에 관한 정보를 표시합니다:

```
$ svn info foo.c
Path: foo.c
Name: foo.c
URL: http://svn.red-bean.com/repos/test/foo.c
Revision: 4417
Node Kind: file
Schedule: normal
Last Changed Author: sally
Last Changed Rev: 20
Last Changed Date: 2003-01-13 16:43:13 -0600 (Mon, 13 Jan 2003)
Text Last Updated: 2003-01-16 21:18:16 -0600 (Thu, 16 Jan 2003)
Properties Last Updated: 2003-01-13 21:50:19 -0600 (Mon, 13 Jan 2003)
Checksum: /3L38YwzhT93BWvgpdF6Zw==
```

디렉토리에 대한 정보를 표시할 수도 있습니다:

```
$ svn info vendors
Path: trunk
URL: http://svn.red-bean.com/repos/test/vendors
Revision: 19
Node Kind: directory
```

Schedule: normal  
 Last Changed Author: harry  
 Last Changed Rev: 19  
 Last Changed Date: 2003-01-16 23:21:19 -0600 (Thu, 16 Jan 2003)

## svn list

### Name

svn list -- 저장소(repository)중의 디렉토리 엔트리를 일람표 가리킵니다.

### 용법

```
svn list [TARGET...]
```

### 설명

각각의 TARGET 파일과 TARGET 디+쿠트리의 저장소(repository) 안의 내용을 일람표 가리킵니다. TARGET 가 작업 카피의 패스인 경우, 대응하는 저장소(repository) URL 가 이용됩니다.

디폴트의 TARGET 치는 '.'로, 현재의 작업 카피 디렉토리 의 저장소(repository) URL를 의미합니다.

--verbose를 사용하면(자) 이하의 필드가 아이템마다의 상태를 나타냅니다:

- 마지막 커밋의 리비전 번호
- 마지막 커밋을 한 사람
- 테이터 사이즈 (아르바이트 단위의)
- 마지막 커밋의 일시

### 별명

ls

### 변경 대상

없음

### 저장소(repository)에의 액세스

발생한다

### 스위치

```
--revision (-r) REV
--verbose (-v)
--recursive (-R)
--username USER
--password PASS
--no-auth-cache
--non-interactive
```

### 예

**svn list** 는, 작업 카피를 다운로드하는 것 없이 어떤 파일이 저장소(repository)에 있을까를 안다 때에 도움이 됩니다:

```
$ svn list http://svn.red-bean.com/repos/test/support
README.txt
INSTALL
예/
```

UNIX 의 **ls**와 같이 추가 정보를 얻기 위해서(때문에) --verbose 스위치를 붙일 수도 있습니다:

```
svn list --verbose file:///tmp/repos
16 sally      28361 Jan 16 23:18 README.txt
27 sally      0 Jan 18 15:27 INSTALL
24 harry      Jan 18 11:27 예/
```

자세하게는>(을)를 봐 주세요.



## svn log

### Name

svn log -- 커밋 로그 메시지의 표시.

### 용법

```
svn log [PATH]
```

```
svn log URL [PATH...]
```

### 설명

디폴트의 타겟은 현재의 작업 디렉토리의 패스가 됩니다. 인수를 지정하지 않으면 **svn log** 는 자신의 작업 카피의 현재의 작업 디렉토리 자신과 그 내부의 모든 파일과 디렉토리에 관한 로그 메시지를 표시합니다. 하나의 패스, 하나 이상의 리비전, 혹은 그러한 편성을 지정하는 것으로 결과 내용을 지정할 수 있습니다. 로컬 패스의 디폴트 리비전 범위는, BASE:1 입니다.

URL만을 지정하면, 그 URL 에 포함되는 모든 로그 메시지가 표시됩니다. URL의 뒤에 패스를 붙이면 URL중의 그러한 패스에 포함된 메시지만이 표시됩니다. URL 의 디폴트 리비전 범위는 HEAD:1 입니다.

--verbose 를 지정하면(자)**svn log** (은)는 각각의 로그 메시지와 함께 관련한 모든 패스를 표시합니다. --quiet 를 지정하면(자) **svn log** (은)는 로그 메시지의 본체 부분을 표시하지 않게 됩니다(이것은 --verbose 스윗치와 양립합니다).

각각의 로그 메시지는, 그 리비전에 영향이 있는 패스가 두 번 이상 요구되어도, 한 번만 표시됩니다. 디폴트에서는 Logs cross copy history; --strict 로 이 동작을 무효로 할 수 있습니다.

### 별명

없음

### 변경 대상

없음

### 저장소(repository)에의 액세스

발생한다

### 스윗치

```
--revision (-r) REV
--quiet (-q)
--verbose (-v)
--targets FILENAME
--username USER
--password PASS
--no-auth-cache
--non-interactive
--strict
--incremental
--xml
```

### 예

최상정도로 **svn log** 를 실행하는 것에 의해 작업 카피중의 변경된 모든 패스의 로그 메시지를 볼 수가 있습니다:

```
$ svn log
```

```
-----
r20 | harry | 2003-01-17 22:56:19 -0600 (Fri, 17 Jan 2003) | 1 line
```

```
Tweak.
```

```
-----
r17 | sally | 2003-01-16 23:21:19 -0600 (Thu, 16 Jan 2003) | 2 lines
```

작업 카피중의 특정의 파일에 관한 모든 로그 메시지를 조사합니다:

```
$ svn log foo.c
```

```
-----
r32 | sally | 2003-01-13 16:43:13 -0600 (Mon, 13 Jan 2003) | 1 line
```

```
Added defines.
```

```
-----
r28 | sally | 2003-01-07 21:48:33 -0600 (Tue, 07 Jan 2003) | 3 lines
```

작업 카피가 수중에 없는 경우, URL를 log 할 수가 있습니다:

```
$ svn log http://svn.red-bean.com/repos/test/foo.c
-----
r32 | sally | 2003-01-13 16:43:13 -0600 (Mon, 13 Jan 2003) | 1 line
Added defines.
-----
r28 | sally | 2003-01-07 21:48:33 -0600 (Tue, 07 Jan 2003) | 3 lines
```

같은 URL아래의 몇개의 다른 패스를 갖고 싶은 경우URL [PATH...] 구문을 사용할 수가 있습니다.

```
$ svn log http://svn.red-bean.com/repos/test/ foo.c bar.c
-----
r32 | sally | 2003-01-13 16:43:13 -0600 (Mon, 13 Jan 2003) | 1 line
Added defines.
-----
r31 | harry | 2003-01-10 12:25:08 -0600 (Fri, 10 Jan 2003) | 1 line
Added new file bar.c
-----
r28 | sally | 2003-01-07 21:48:33 -0600 (Tue, 07 Jan 2003) | 3 lines
```

그것은 커멘트 라인상에서 양쪽 모두의 URL를 명시적으로 두는 것과 같은 것이 됩니다:

```
$ svn log http://svn.red-bean.com/repos/test/foo.c W
      http://svn.red-bean.com/repos/test/bar.c
```

복수의 log 커멘트의 결과를 연결하고 싶은 경우, --incremental 스위치를 사용할 수가 있습니다. **svn log**는 보통 메시지의 최초로 데쉬의 행을 표시 해, 각각의 계속하는 로그 메시지를 표시해, 마지막 로그 메시지가 거기에 계속됩니다. 만약 **svn log** 를 두 개의 리비전 범위에서 실행했을 경우, 다음과 같은 출력 (이)가 됩니다:

```
$ svn log -r 14:15
-----
r14 | ...
-----
r15 | ...
-----
```

그러나, 두 차례로 되지 않은 로그 메시지를 파일에 출력하고 싶은 경우, 무엇인가 다음과 같은 기분이 들겠지요:

```
$ svn log -r 14 > mylog
$ svn log -r 19 >> mylog
$ svn log -r 27 >> mylog
$ cat mylog
-----
r14 | ...
-----
-----
r19 | ...
-----
-----
r27 | ...
-----
```

incremental 스위치를 사용하면, 출력중의 중복 한 데쉬행 의 표시를 피할 수가 있습니다:

```
$ svn log --incremental -r 14 > mylog
$ svn log --incremental -r 19 >> mylog
$ svn log --incremental -r 27 >> mylog
$ cat mylog
-----
r14 | ...
-----
```

```
r19 | ...
```

```
r27 | ...
```

--incremental 스윗치는, --xml 스윗치를 사용했을 때 것과 같은 출력 제어를 합니다.

**Tip:** 만약 **svn log** 를 특정의 패스상의 특정의 리비전 지정으로 실행하면(자) 아무것도 출력되지 않을 것입니다

```
$ svn log -r 20 http://svn.red-bean.com/untouched.txt
```

그것은 패스는 리비전에 따라서는 수정되지 않았던 것을 의미 하고 있습니다. 저장소(repository)의 최상정도로 log 하는지, 그 리비전으로 수정한 파일을 알고 있다면, 명시적으로 그것을 지정하는 것이 할 수 있습니다:

```
$ svn log -r 20 touched.txt
```

```
r20 | sally | 2003-01-17 22:56:19 -0600 (Fri, 17 Jan 2003) | 1 line
```

```
Made a change.
```

## svn merge

### Name

svn merge -- 두 소스의 차이를 작업 카피 패스에 반영합니다.

### 용법

```
svn merge sourceURL1[@N] sourceURL2[@M] [WCPATH]
```

```
svn merge -r N:M SOURCE [PATH]
```

### 설명

최초의 형식에서는 소스가 되는 URL는 리비전 N 와 M 로 지정되어 그 둘이 비교됩니다. 리비전이 생략 되면 HEAD 를 지정된 것과 간주합니다.

두번째의 형식에서는 SOURCE 는 URL 인가, 작업 카피 아이템이며, 그 경우, 대응한 URL가 이용됩니다. 이, 리비전 N 와 M 의 URL가, 비교 대상과 됩니다.

WCPATH 가 변경을 받아들이는 작업 카피 패스입니다. 만약 WCPATH가 생략 되면(자), 디폴트로써, '.'가 이용됩니다. 다만, 양쪽 모두의 소스의 베이스명이 같아, 게다가 그 이름의 파일이'. '에 있는 경우는 별개로, 이 경우는, 차분은 그 파일에 적용됩니다.

### 별명

없음

### 변경 대상

작업 카피

### 저장소(repository)에의 액세스

URL에 대해서 동작할 때 마서 발생한다

### 스윗치

```
--revision (-r) REV
--non-recursive (-N)
--quiet (-q)
--force
--dry-run
--username USER
--password PASS
--no-auth-cache
--non-interactive
```

### 예

브랜치(branch)를 주계에 merge 합니다 (주계의 작업 카피가 있는 것이 전제가 됩니다):

```
$ svn merge http://svn.red-bean.com/repos/trunk/vendors W
      http://svn.red-bean.com/repos/branches/vendors-with-fix
U   myproj/tiny.txt
U   myproj/thhgttg.txt
U   myproj/win.txt
U   myproj/flo.txt
```

리비전 23으로 분기(브랜치(branch)화) 해, 그 브랜치(branch)안의 주제로 변경을 merge 하고 싶다고 합니다. 이것에는 브랜치(branch)의 작업 카피 중(안)에서 이하와 같은 조작을 합니다:

```
$ svn merge -r 23:30 file:///tmp/repos/trunk/vendors
U   myproj/thhgttg.txt
```

변경을 하나의 파일에 merge 하려면:

```
$ cd myproj
$ svn merge -r 30:31 thhgttg.txt
U   thhgttg.txt
```

## svn mkdir

### Name

svn mkdir -- 버전 관리하에 있는 새로운 디렉토리를 만듭니다.

### 용법

```
svn mkdir PATH...
```

```
svn mkdir URL...
```

### 설명

PATH 또는 URL를 마지막 부분으로 할 것 같은 디렉토리를 만듭니다. 작업 카피 PATH 로 지정된 디렉토리는 작업 카피에의 추가로서 예고됩니다. URL 에 의해 지정된 디렉토리는 작성과 동시에 커밋됩니다. 복수의 디렉토리 URL 는 불분할에 커밋됩니다. 어느 쪽의 경우에서도 도중의 디렉토리는 모두 존재하고 있지 않으면 안됩니다.

### 별명

없음

### 변경 대상

작업 카피. URL를 지정했을 경우는 저장소(repository)

### 저장소(repository)에의 액세스

URL를 지정했을 경우는 발생한다

### 스위치

```
--message (-m) TEXT
--file (-F) FILE
--quiet (-q)
--username USER
--password PASS
--no-auth-cache
--non-interactive
--encoding ENC
--force-log
```

### 례

작업 카피중에 디렉토리를 만든다:

```
$ svn mkdir newdir
A      newdir
```

저장소(repository)에 만든다(커밋이 발생하므로 로그 메시지를 지정할 필요가 있다) :

```
$ svn mkdir -m "Making a new dir. " http://svn.red-bean.com/repos/newdir
```

Committed revision 26.

## svn move

### Name

svn move -- 파일이나 디렉토리를 이동한다.

### 용법

```
svn move SRC DST
```

### 설명

이 커멘트는 작업 카피 또는 저장소(repository)에 있다 파일이나 디렉토리를 이동합니다.

**Tip:** 이 커멘트는 **svn copy** 의 뒤에 **svn delete**를 실행하는 것과 같은 것입니다.

#### Warning

Subversion에서는 작업 카피와 URL의 사이의 이동은 서포트하고 있지 않습니다. 게다가 파일의 이동은 하나의 저장소(repository)의 내부에서만 가능합니다 Subversion은 저장소(repository)간의 이동을 서포트하고 있지 않습니다.

WC - WC

이동하고 나서, 파일이나 디렉토리를 추가 예고합니다.

URL - URL

서버상에서의 명칭 변경

### 별명

mv, rename, ren

### 변경 대상

작업 카피, URL를 지정했을 경우는 저장소(repository)

### 저장소(repository)에의 액세스

URL를 지정했을 경우는 발생한다

### 스윗치

```
--message (-m) TEXT
--file (-F) FILE
--revision (-r) REV
--quiet (-q)
--force
--username USER
--password PASS
--no-auth-cache
--non-interactive
--encoding ENC
--force-log
```

### 예

작업 카피의 파일을 이동:

```
$ svn move foo.c bar.c
A      bar.c
D      foo.c
```

저장소(repository)의 파일을 이동(커밋이 발생하므로 커밋 메시지를 지정할 필요가 있다):

```
$ svn move -m "Move a file" http://svn.red-bean.com/repos/foo.c W
http://svn.red-bean.com/repos/bar.c
```

Committed revision 27.

## svn propdel

### Name

svn propdel --아이템으로부터 속성을 삭제합니다.

### 용법

```
svn propdel PROPNAME [PATH...]
```

```
svn propdel PROPNAME --revprop -r REV [URL]
```

### 설명

이것은 파일, 디렉토리, 리비전으로부터 속성을 삭제합니다. 최초의 형식은 작업 카피의 버전 관리된 속성을 삭제해, 두번째의 형식에서는 저장소(repository) 리비전상의 버전 관리되어 있지 않은 속성을 삭제합니다.

### 별명

pdel, pd

### 변경 대상

작업 카피. URL를 지정했을 경우는 저장소(repository)

### 저장소(repository)에의 액세스

URL를 지정했을 경우만 발생한다

### 스위치

```
--quiet (-q)  
--recursive (-R)  
--revision (-r) REV  
--revprop
```

### 예

작업 카피의 파일로부터 속성을 삭제한다

```
$ svn propdel svn:mime-type some-script  
property 'svn:mime-type' deleted from 'some-script'.
```

리비전 속성을 삭제한다:

```
$ svn propdel --revprop -r 26 release-date  
property 'release-date' deleted from repository revision '26'
```

## svn propedit

### Name

svn propedit --버전 관리되고 있는 하나 이상의 아이템의 속성을 편집한다.

### 용법

```
svn propedit PROPNAME PATH...
```

```
svn propedit PROPNAME --revprop -r REV [URL]
```

### 설명

하나 이상의 속성을 좋아하는 에디터로 수정합니다. 최초의 형식은 작업 카피중의 버전 관리된 속성을 편집합니다. 두번째의 형식에서는 저장소(repository) 리비전상의 버전 관리되어 있지 않다 속성을 편집합니다.

## 별명

pedit, pe

## 변경 대상

작업 카피. URL를 지정했을 경우는 저장소(repository)

## 저장소(repository)에의 액세스

URL를 지정했을 경우만 발생한다

## 스위치

```
--revision (-r) REV
--revprop
--encoding ENC
```

## 예

**svn propedit** 는, 복수의 값을 가진다 속성을 간단하게 변경할 수가 있습니다:

```
$ svn propedit svn:keywords foo.c
svn will launch your favorite editor here, with a buffer open
containing the current contents of the svn:keywords property. You
can add multiple values to a property easily here by entering one
value per line.
Set new value for property 'svn:keywords' on 'foo.c'
```

## svn propget

### Name

svn propget -- 속성의 값을 표시합니다.

### 용법

```
svn propget PROPNAME [PATH...]
```

```
svn propget PROPNAME --revprop -r REV [URL]
```

### 설명

파일, 디렉토리, 리비전의 속성치를 표시합니다. 최초의 형식은 작업 카피에 있는 하나 이상의 아이템의 버전 관리된 속성을 표시합니다. 두번째의 형식인 저장소(repository) 리비전의 버전 관리하고 있지 않는 속성을 표시합니다. 속성에 대한 자세한 것은 > (을)를 봐 주세요.

## 별명

pget, pg

## 변경 대상

작업 카피. URL를 지정했을 경우는 저장소(repository)

## 저장소(repository)에의 액세스

URL를 지정했을 때 마서 발생한다

## 스위치

```
--recursive (-R)
--revision (-r) REV
--revprop
```

## 예

작업 카피중의 아이템의 속성을 조사한다:

```
$ svn propget svn:keywords foo.c
Author
Date
Rev
```

리비전 속성에 대해서도 같이:

```
$ svn propget svn:log --revprop -r 20
Began journal.
```

## svn proplist

### Name

svn proplist -- 모든 속성을 일람표 가리킵니다.

### 용법

```
svn proplist [PATH...]
```

```
svn proplist --revprop -r REV [URL]
```

### 설명

파일, 디렉토리, 리비전의 모든 속성을 일람표 가리킵니다. 최초의 형식에서는 작업 카피중의 버전 관리된 속성을 표시합니다만, 두번째의 형식인 저장소(repository) 리비전의 속성을 표시합니다.

### 별명

plist, pl

### 변경 대상

작업 카피. URL가 지정되어 있는 경우는 저장소(repository)

### 저장소(repository)에의 액세스

URL를 지정했을 경우만 발생한다

### 스위치

```
--verbose (-v)
--recursive (-R)
--revision (-r) REV
--revprop
```

### 예

작업 카피의 아이템의 속성의 일람을 보고 싶은 경우는 proplist 커멘드를 사용할 수가 있습니다:

```
$ svn proplist foo.c
Properties on 'foo.c':
  svn:mime-type
  svn:keywords
  owner
```

그러나--verbose 플래그를 붙이면(자), 각각의 속성의 값도 함께 표시할 수가 있으므로 매우 편리합니다:

```
$ svn proplist --verbose foo.c
Properties on 'foo.c':
  svn:mime-type : text/plain
  svn:keywords : Author Date Rev
  owner : sally
```

## svn propset



## Name

svn propset -- 파일, 디렉토리, 리비전의 PROPNAME 의 값 (을)를 PROPVAL로 설정한다.

## 용법

```
svn propset PROPNAME [PROPVAL | -f VALFILE] PATH [PATH [PATH ... ]]
```

```
svn propset PROPNAME --revprop -r REV [PROPVAL | -f VALFILE] [URL]
```

## 설명

파일, 디렉토리, 리비전의 PROPNAME의 값을 PROPVAL로 설정합니다. 최초의 예는 버전 관리된 작업 카피중의 속성치의 변경으로, 두 번째는 버전 관리되어 있지 않은 저장소(repository)상의 리비전 속성치의 작성입니다.

**Tip:** Subversion은, 동작에 영향을 주는 많은 "특별한"속성 (을)를 갖고 있습니다. 자세하게는 >(을)를 봐 주세요.

## 별명

pset, ps

## 변경 대상

작업 카피. URL를 지정했을 경우만 저장소(repository)

## 저장소(repository)에의 액세스

URL를 지정했을 경우만 발생한다

## 스위치

```
--file (-F) FILE
--quiet (-q)
--revision (-r) REV
--targets FILENAME
--recursive (-R)
--revprop
--encoding ENC
```

## 예

파일의 MIME 타입을 설정한다:

```
$ svn propset svn:mime-type image/jpeg foo.jpg
property 'svn:mime-type' set on 'foo.jpg'
```

UNIX 상에서, 어느 파일에 실행 속성을 붙이고 싶을 때에는:

```
$ svn propset svn:executable ON somescript
property 'svn:executable' set on 'somescript'
```

아마, 공동 작업자의 편의를 생각하면(자), 어느 속성을 설정 하려면 내부적인 폴리스가 없으면 안됩니다:

```
$ svn propset owner sally foo.c
property 'owner' set on 'foo.c'
```

특정의 리비전의 로그 메시지를 틀러 버려, 그것을 변경하고 싶을 때, --revprop 를 사용해, svn:log 에 새로운 메시지를 설정합니다:

```
$ svn propset --revprop -r 25 svn:log "Journalled about trip to New York. "
property 'svn:log' set on repository revision '25'
```

혹은, 작업 카피를 가지고 있지 않은 경우에서도, URL를 설정하는 것이 할 수 있습니다.

```
$ svn propset --revprop -r 26 svn:log "Document nap. " http://svn.red-bean.com/repos
property 'svn:log' set on repository revision '25'
```

마지막으로, 속성치를 파일을 입력으로서 설정할 수도 있습니다. 이 방법으로, 속성치에 바이너리치를 설정하는 것 조차 할 수 있습니다:

```
$ svn propset owner-pic -F sally.jpg moo.c
property 'owner-pic' set on 'moo.c'
```

#### Warning

디폴트에서는 Subversion 저장소(repository)중의 리비전 속성은 변경 할 수 없습니다. 저장소(repository) 관리자는 'pre-revprop-change'라고 한 다 이름의 훅을 만드는 것으로 명시적으로 리비전 프로퍼티의 수정을 유효하게 하지 않으면 안됩니다. 폭스 클립트에 대해 자세한 것은 > (을)를 봐 주세요.

## svn resolved

### Name

svn resolved -- 작업 카피의 파일 또는 디렉토리의 "충돌" 상태를 잡기 들여다 봅니다.

### 용법

```
svn resolved PATH...
```

### 설명

작업 카피의 파일 또는 디렉토리의 "충돌" 상태를 잡기 들여다 봅니다. 이 커멘드는 충돌 마커를 의미적으로 해소하는 것이 아니라, 단지 충돌에 관계한 중간 파일을 삭제해, PATH로 한번 더 커밋한다 뿐입니다. 즉 Subversion에 그 충돌은 벌써 "해소되었다"라고 전합니다. 충돌의 해소에 대한 자세한 것은 > (을)를 봐 주세요.

### 별명

없음

### 변경 대상

작업 카피

### 저장소(repository)에의 액세스

No

### 스위치

```
--targets FILENAME
--recursive (-R)
--quiet (-q)
```

### 예

갱신 조작중에 충돌이 있었을 경우, 작업 카피는 세 개의 새로운 파일을 만듭니다:

```
$ svn update
C foo.c
Updated to revision 31.
$ ls
foo.c
foo.c.mine
foo.c.r30
foo.c.r31
```

충돌을 해소해,foo.c 의 커밋의 준비가 할 수 있던 상태에 있는 경우,**svn resolved** 는 당신의 작업 카피에, 주의하지 않으면 안 되는 모든 일을 전합니다.

#### Warning

단지 충돌 파일을 삭제하고 나서 커밋할 수도 **있습니다** 하지만 **svn resolved** 는 작업 카피 관리 area의 기록으로서 충돌 파일을 삭제했다 일도 덧붙이므로, 이 커멘드를 사용하는 것을 추천합니다.

## svn revert

### Name

svn revert -- 로컬 파일에의 모든 편집을 취소합니다.

## 용법

```
svn revert PATH...
```

## 설명

파일, 디렉토리에 대한 변경을 모두 취소해 충돌 상태를 해소합니다. **svn revert** 작업 카피의 아이템의 내용 만이 아니고, 속성의 변경도 취소합니다. 한층 더 벌써 한 예고 조작을 취소하는데도 사용할 수 있습니다. (예를 들어, 파일에 대한 추가 또는 삭제의 예고 도"삭제되

## 별명

없음

## 변경 대상

작업 카피

## 저장소(repository)에의 액세스

없음

## 스위치

```
--targets FILENAME
--recursive (-R)
--quiet (-q)
```

## 예

파일에 대한 변경의 취소:

```
$ svn revert foo.c
Reverted foo.c
```

디렉토리 전체를 취소하고 싶은 경우는, **--recursive** 플래그를 사용합니다:

```
$ svn revert --recursive .
Reverted newdir/afile
Reverted foo.c
Reverted bar.txt
```

마지막으로, 어느 예고 조작도 취소할 수가 있습니다:

```
$ svn add mistake.txt whoops
A      mistake.txt
A      whoops
A      whoops/oopsie.c

$ svn revert mistake.txt whoops
Reverted mistake.txt
Reverted whoops

$ svn status
?      mistake.txt
?      whoops
```

### Warning

**svn revert** 의 대상을 지정하지 않으면, 그것은 아무것도 하지 않습니다 잘못된 수정의 파기로부터 작업 카피를 지키기 위해서, **svn revert** 조작은 적어도 하나의 인수를 지정하도록(듯이) 요구합니다.

## svn status

### Name

svn status -- 작업 카피에 있는 파일이나 디렉토리 상태를 표시.

## 용법

```
svn status [PATH...]
```

## 설명

작업 카피에 있는 파일이나 디렉토리 상태를 표시합니다. 인수가 없는 경우는, 로컬로 수정된 아이템만이 표시됩니다(저장소(repository)에 대하는 액세스는 발생하지 않습니다 --show-updates를 사용하면(자), 작업 리비전과 서버의 최신이 아닌 정보도 추가됩니다. --verbose를 사용하면(자), 모든 아이템에 대한다 완전한 리비전 정보를 표시합니다.

출력의 최초의 5열의 칼럼은 각각 한 글자폭으로, 작업 카피 아이템 마다 여러가지 정보를 표시합니다.

최초의 칼럼은, 아이템이 추가, 삭제, 그 이외의 변경, 목 상태인지를 (을)를 나타냅니다.

```
' '
```

변경은 없습니다.

```
'A'
```

아이템은 추가 예고되고 있습니다.

```
'D'
```

아이템은 삭제 예고되고 있습니다.

```
'M'
```

아이템은 수정되었습니다.

```
'C'
```

아이템은 저장소(repository)로부터 받은 갱신에 의해 충돌 한 상태에 있습니다.

```
'I'
```

아이템은 무시되고 있는 속성입니다 (예를 들어, svn:ignore와 같은)

```
'?'
```

아이템은 버전 관리하에 없습니다.

```
'!'
```

아이템은 없어졌습니다(이것은 예를 들어, **svn**를 사용하지 않고 파일을 삭제하거나 이동했을 경우에 일어납니다). 또, 이것은 디렉토리가 불완전한 것을 나타내고 있습니다 (체크아웃이나 갱신이 중단된, 등).

```
'~'
```

아이템은 디렉토리로서 버전 관리되어 있습니다만, 파일로 옮겨놓을 수 있어 버렸습니다. 혹은 그 역의 상황이 일어났습니다.

두번째의 칼럼은 파일이나 디렉토리의 속성 상태를 나타냅니다.

```
' '
```

수정은 없습니다.

```
'M'
```

이 아이템의 속성은 수정되었습니다.

```
'C'
```

이 아이템의 속성은 저장소(repository)로부터 받은 속성 갱신에 의해 충돌한 상태에 있습니다.

3번째의 칼럼은 작업 카피가 락 되고 있는 경우에만 사용됩니다.

```
' '
```

아이템은 락 되고 있지 않습니다.

```
'L'
```

아이템은 락 되고 있습니다.

4번째의 칼럼은 아이템이 추가 예고되고 있는 경우에만 사용됩니다.

```
' '
```

커밋 기다리는 예고는 없습니다.

```
'+'
```

커밋 기다리는 예고가 있습니다.

다섯번째의 칼럼은 아이템이 부모에 대해서 상대적으로 교체되었다 때에만 사용됩니다. (>를 봐 주세요).

```
' '
```

아이템은 친디렉토리의 아이입니다.

```
'S'
```

아이템은 바뀌고 있습니다.

최신 상태에 관계한 정보가 8번째의 칼럼에 표시됩니다 (--show-updates 스위치를 건네주었을 경우).

```
' '
```

작업 카피의 아이템은 최신입니다.

```
'*'
```

서버에는 아이템의 것 좀 더 새로운 버전이 존재합니다.

나머지의 필드는 공백에서 단락지어진 가변장입니다. `--show-updates` 나 `--verbose` (을)를 지정했을 경우는 작업 리비전이 다음의 필드가 됩니다.

`--verbose` 스위치를 지정하면(자) 마지막 커밋 리비전 (와)과 그것을 한 사람이 다음에 표시됩니다.

작업 카피의 패스는 항상 마지막 필드가 되므로, 공백을 포함하는 것이 할 수 있습니다.

## 별명

stat, st

## 변경 대상

없음

## 저장소(repository)에의 액세스

`--show-updates`가 지정되었을 경우에 한발생한다

## 스위치

```
--show-updates (-u)
--verbose (-v)
--non-recursive (-N)
--quiet (-q)
--username USER
--password PASS
--no-auth-cache
--non-interactive
--force-log
--no-ignore
```

## 예

작업 카피(?변경점을 조사하기 위한 제일 간단한 방법:

```
$ svn status wc
M      wc/bar.c
A +    wc/qax.c
```

작업 카피중, 어느 파일이 최신이 아닌가를 알고 싶은 경우 (은)는 `--show-updates` 스위치를 지정해 주세요(이것은 작업 카피의 내용을 **결**코 변경하지 않습니다). 마지막에 자신의 작업 카피를 갱신하고 나서 저장소(repository)의 `wc/foo.c` (으)로 변경이 있었을 때에는 다음과 같이 됩니다:

```
$ svn status --show-updates wc
M          965      wc/bar.c
*          965      wc/foo.c
A +        965      wc/qax.c
Head revision:  981
```

### Warning

`--show-updates`는, 최신이 아닌 아이템의 이웃에 asterisk를 두는 **만**입니다(즉, **svn update**를 실행했다고 하면 저장소(repository)로부터 의 정보로 갱신될 아이템의 전에게만, 이라고 하는 의미입니다). `--show-updates` 은, 아이템의 저장소(repository) 버전을 반영 한 상태 일람을 표시하는 것은 **아닙니다**.

마지막으로, status 서브 커맨드로 제일 많은 정보를 얻으려면:

```
$ svn status --show-updates --verbose wc
M          965      938 sally      wc/bar.c
*          965      922 harry      wc/foo.c
A +        965      687 harry      wc/qax.c
          965      687 harry      wc/zig.c
Head revision:  981
```

**svn status** 의 것 좀 더 많은 예는 >에 있습니다.

## svn switch

### Name

svn switch -- 작업 카피를 다른 URL에 갱신합니다.

### 용법

```
svn switch URL [PATH]
```

### 설명

이 서브 커멘드는 자신의 작업 카피를 새로운 URL에 갱신해, 복제를 만듭니다. 그렇게 할 필요는 없습디다만, 보통은 그 URL는 바탕으로 되는 작업 카피와 공통의 조상을 가집니다. 이것이 Subversion로 작업 카피를 다른 브랜치(branch)에 이동시키는 방법 입니다. > 에 의해 자세한 설명이 있습니다.

### 별명

sw

### 변경 대상

작업 카피

### 저장소(repository)에의 액세스

발생한다

### 스위치

```
--revision (-r) REV
--non-recursive (-N)
--quiet (-q)
--diff3-cmd CMD
--relocate
--username USER
--password PASS
--no-auth-cache
--non-interactive
```

### 예

지금'vendors-with-fix'로부터 분기 한'vendors'라고 하는 디렉토리의 내부에 있어, 그 브랜치(branch)의 작업 카피로 옮기고 싶을 때에는:

```
$ svn switch http://svn.red-bean.com/repos/branches/vendors-with-fix .
U myproj/foo.txt
U myproj/bar.txt
U myproj/baz.c
U myproj/qux.c
Updated to revision 31.
```

그리고, 원래대로 돌아가고 싶을 때에는, 최초로 작업 카피를 체크아웃 했다 저장소(repository)의 장소를 URL로서 지정할 뿐입니다:

```
$ svn switch http://svn.red-bean.com/repos/trunk/vendors .
U myproj/foo.txt
U myproj/bar.txt
U myproj/baz.c
U myproj/qux.c
Updated to revision 31.
```

**Tip:** 작업 카피 전체를 바꾸고 싶지 않은 경우, 그 일부만을 브랜치(branch)에 바꿀 수도 있습니다.

저장소(repository)의 장소가 변경되었지만, 그대로 계속해 이용하고 싶다고 생각한다 작업 카피가 있는 경우,**svn switch --relocate**를 사용해 어느 URL로부터 다른 URL를 가리키도록(듯이) 작업 카피를 변경할 수가 있습니다:

```
$ svn checkout file:///tmp/repos test
A test/a
A test/b

$ mv repos newlocation
$ cd test/
```

```
$ svn update
svn: Couldn't open a repository.
svn: Unable to open an ra_local session to URL
svn: Unable to open repository 'file:///tmp/repos'

$ svn switch --relocate file:///tmp/repos file:///tmp/newlocation .
$ svn update
At revision 3.
```

## svn update

### Name

svn update -- 작업 카피의 갱신.

### 용법

```
svn update [PATH...]
```

### 설명

**svn update** 는, 저장소(repository)의 수정을 작업 카피에 반영합니다. 리비전을 지정하지 않으면, HEAD 리비전의 최신 의 내용이 반영됩니다. 그렇지 않으면, --revision 스위치 그리고 지정된 리비전에 작업 카피를 동기 합니다.

갱신되는 아이템 마다, 어떠한 동작을 일으켰는지를 나타내는 캐릭터로 시작된다 행이 표시됩니다. 이 캐릭터는 이하와 같은 의미입니다:

```
A
    추가
D
    삭제
U
    갱신
C
    충돌
M
    merge
```

최초의 칼럼의 캐릭터는 실제의 파일의 갱신을 나타냅니다만, 파일의 속성의 갱신 상황은 두번째의 칼럼에서 가리킵니다.

### 별명

up

### 변경 대상

작업 카피

### 저장소(repository)에의 액세스

발생한다

### 스위치

```
--revision (-r) REV
--non-recursive (-N)
--quiet (-q)
--username USER
--password PASS
--no-auth-cache
--non-interactive
```

### 예

마지막 갱신 후에 일어난 저장소(repository)의 수정을 수중에 넣습니다:

```
$ svn update
A newdir/toggle.c
A newdir/disclose.c
A newdir/launch.c
```

```
D newdir/README
Updated to revision 32.
```

좀 더 낡은 리비전으로 작업 카피를 갱신할 수도 있습니다. (Subversion 는 CVS 와 같이 "붙어" 파일 의 개념을 가지지 않습니다. >(을)를 봐 주세요):

```
svn update -r30
A newdir/README
D newdir/toggle.c
D newdir/disclose.c
D newdir/launch.c
U foo.c
Updated to revision 30.
```

**Tip:** 하나의 파일의 낡은 버전을 조사하고 싶을 때에는 **svn cat**를 사용하고 싶어질지도 알려지지 않습니다.

## 1.2. svnadmin

**svnadmin** 는 Subversion 저장소(repository)를 감시하거나 수복하거나하기 위한 관리툴입니다. 자세한게는>을 봐 주세요.

**svnadmin**는 직접 저장소(repository)에 대한 액세스에 보다 동작하므로(그리고, 저장소(repository)가 존재하는 머신상에서만 이용하는 것이 할 수 있으므로), URL가 아니고, 패스명에 의해 저장소(repository)를 참조합니다.

### 1.2.1. svnadmin스윗치

**--bypass-hooks**

저장소(repository)의 훅을 우회 합니다.

**--copies**

패스를 조사할 때 카피 히스토리를 표시합니다.

**--in-repos-template ARG**

새로운 저장소(repository)를 작성할 경우에, 저장소(repository) 구조의 템플릿을 지정합니다.

"in-repository" 템플릿은 저장소(repository) 자신의 레이아웃 (을)를 지정합니다(그것은, db/ 디렉토리안의 Berkeley DB 파일 그리고 사용됩니다). /trunk, /branches ... (와)과 같은 형태입니다. 템플릿은 저장소(repository)의 최초의 로드를(hook 없음으로) 실행하는데 관리자나 어플리케이션에 의해 사용하는 것이라고 옵니다. 디폴트는 없습니다. 아무것도 지정하지 않으면 저장소(repository)는 "빈 상태로" 로 작성됩니다.

**--incremental**

리비전의 전체를 덤프 하는 것이 아니라, 이전의 리비전에 대한 차분으로서 덤프 합니다.

**--on-disk-template ARG**

만들고 싶다고 생각하는 저장소(repository)를 위해서(때문에), 디스크상의 구조를 템플릿에 이용하도록(듯이) 지정합니다.

"on-disk" 템플릿은 저장소(repository)의 디렉토리 그것을 나타냅니다. 각각의 템플릿은 이름을 갖고, "default" 의 디스크 템플릿은, 이하와 같은 내용을 가집니다:

- default/
- README.txt
- dav/
- format
- hooks/
- post-commit.tmpl
- post-revprop-change.tmpl
- pre-commit.tmpl
- pre-revprop-change.tmpl
- start-commit.tmpl
- locks/
- db.lock

디스크상의 구조의 전형적인 사용법은 작성되는 폭스 클립트로서 벌써 정의된 것을 사용하는 것 같은 경우입니다. 예를 들어 커밋 시의 메일러와 백업 스크립트가 있다 post-commit 스크립트를 만들어 있다고 합니다. 이후, 관리자가 새로운 저장소(repository)를 만들 때마다, 모든 훅을 미리 자동적으로 포함한, 이 새로운 템플릿을 지정한다 일이 생깁니다.

**--revision (-r) ARG**

조작 대상이 되는 특정의 리비전을 지정합니다.

### 1.2.2. svnadmin 서브 커맨드



**Table of Contents**

[svnadmin list-unused-dblogs](#) -- 어느 로그 파일이 안전하게 삭제 가능한가를 Berkeley DB 에 문의 .  
[svnadmin create](#) -- REPOS\_PATH에 신규의 빈 저장소(repository)를 만듭니다.  
[svnadmin dump](#) -- 파일 시스템의 내용을 표준 출력에 덤프 합니다.  
[svnadmin help](#) --  
[svnadmin load](#) -- 표준 입력으로부터"덤프 형식" 의 데이터를 읽어들입니다.  
[svnadmin lstxns](#) -- 커밋되어 있지 않은 모든 트랜잭션(transaction)의 이름의 표시.  
[svnadmin recover](#) -- 저장소(repository)중이 없어진 상황의 복원.  
[svnadmin rmtxns](#) -- 저장소(repository)로부터 트랜잭션(transaction)를 삭제합니다.  
[svnadmin setlog](#) -- 리비전에 로그 메시지를 설정합니다.

## svnadmin list-unused-dblogs

### Name

svnadmin list-unused-dblogs -- 어느 로그 파일이 안전하게 삭제 가능한가를 Berkeley DB 에 문의 .

### 용법

```
svnadmin list-unused-dblogs REPOS_PATH
```

### 설명

Berkeley DB 는 긴급시의 리커버리를 위해서(때문에) 저장소(repository)의 모든 변경점의 로그를 만듭니다. 시간과 함께 로그 파일은 축적됩니다만, 그 대부분은 벌써 이용되지 않고, 디스크 area를 펼치기 위해서(때문에) 삭제할 수가 있습니다. 자세한 것은 >(을)를 봐주세요.

### 예

모든 이용되어 있지 않은 로그 파일을 저장소(repository)로부터 삭제합니다:

```
$ svnadmin list-unused-dblogs /path/to/repos
/path/to/repos/log. 0000000031
/path/to/repos/log. 0000000032
/path/to/repos/log. 0000000033

$ svnadmin list-unused-dblogs /path/to/repos | xargs rm
## disk space reclaimed!
```

## svnadmin create

### Name

svnadmin create -- REPOS\_PATH에 신규의 빈 저장소(repository)를 만듭니다.

### 용법

```
svnadmin create REPOS_PATH
```

### 설명

지정한 패스에 신규에 빈 저장소(repository)를 만듭니다. 패스가 존재하지 않으면 자동적으로 만들어집니다.

### 스위치

```
--on-disk-template arg
--in-repos-template arg
```

### 예

새롭게 저장소(repository)를 만들려면 단지 이하와 같이 합니다:

```
$ svnadmin create /usr/local/svn/repos
```

## svnadmin dump

### Name

svnadmin dump -- 파일 시스템의 내용을 표준 출력에 덤프 합니다.

## 용법

```
svnadmin dump REPOS_PATH [-r LOWER[:UPPER]] [--incremental]
```

## 설명

파일 시스템의 내용을'dumpfile'가반가능 형식에서 덤프 해, 진행 상황 (을)를 표준 출력에 표시합니다. 리비전 LOWER 로부터, UPPER 까지를 덤프 합니다. 리비전이 지정 되지 않으면 모든 리비전 트리를 덤프 합니다. LOWER만이 지정되었을 경우는 하나의 리비전 트리 만 덤프 합니다. 실제의 사용법에 대해서는> 을 봐 주세요.

## 스위치

```
--revision (-r)
--incremental
```

## 예

저장소(repository) 전체의 덤프:

```
$ svnadmin dump /usr/local/svn/repos
SVN-fs-dump-format-version: 1
Revision-number: 0
* Dumped revision 0.
Prop-content-length: 56
Content-length: 56
```

저장소(repository)의 하나의 트랜잭션(transaction)의 차분 덤프:

```
$ svnadmin dump /usr/local/svn/repos -r 21 --incremental
* Dumped revision 21.
SVN-fs-dump-format-version: 1
Revision-number: 21
Prop-content-length: 101
Content-length: 101
```

# svnadmin help

## Name

svnadmin help

## 용법

```
svnadmin help [SUBCOMMAND...]
```

## 설명

이 서브 커맨드는 네트워크 접속이 잘 되지 않게 되어 이 책의 카피를 읽을 수 없게 되거나 해 어디엔가 헤매어 버렸다 같은 때에 도움이 됩니다.

# svnadmin load

## Name

svnadmin load -- 표준 입력으로부터"덤프 형식" 의 데이터를 읽어들입니다.

## 용법

```
svnadmin load REPOS_PATH
```

## 설명

표준 입력으로부터"덤프 형식" 의 데이터를 읽어들여, 저장소(repository)의 파일 시스템에 새롭다 리비전을 커밋합니다. 진행 상황은 표준 출력에 표시됩니다.

## 예

이것은 백업파일로부터 저장소(repository)를 로드하는 최초의 부분입니다(물론 미리 **svn dump**로 덤프 해 둔 것입니다):

```
$ svnadmin load /usr/local/svn/restored repos-backup
Started new txn, based on original revision 1
* adding path : test ... done.
* adding path : test/a ... done.
```

## svnadmin lstxns

### Name

svnadmin lstxns -- 커밋되어 있지 않은 모든 트랜잭션(transaction)의 이름의 표시.

### 용법

```
svnadmin lstxns REPOS_PATH
```

### 설명

커밋되어 있지 않은 모든 트랜잭션(transaction)의 이름을 표시합니다. 커밋되어 있지 않은 트랜잭션(transaction)는 어떻게 만들어져 어떻게 취급해야할 것인가에 대해서는 > 을 봐 주세요.

### 예

저장소(repository)중의 미해결의 트랜잭션(transaction) 일람의 표시.

```
$ svnadmin lstxns /usr/local/svn/repos/
1w
1x
```

## svnadmin recover

### Name

svnadmin recover -- 저장소(repository)중이 없어진 상황의 복원.

### 용법

```
svnadmin recover REPOS_PATH
```

### 설명

저장소(repository)는 수복될 필요가 있다고 하는 에러 메시지를 받아들었다 때로는 이 커멘트를 실행해 주세요.

| Warning                                                                                                                                              |
|------------------------------------------------------------------------------------------------------------------------------------------------------|
| 저장소(repository)에 액세스 하고 있는 것이 자신 뿐이라고 하는 것이 <b>완전하게 확실한 경우에만</b> 이 커멘트를 실행해 주세요이 커멘드에는 배타적인 액세스권이 필요합니다. 저장소(repository)의 수복에 관한 자세한 것은 >(을)를 봐 주세요. |

### 예

행 해 버린 저장소(repository)의 수복:

```
$ svnadmin recover /usr/local/svn/repos/
Acquiring exclusive lock on repository db.
Recovery is running, please stand by...
Recovery completed.
The latest repos revision is 34.
```

## svnadmin rmtxns

### Name

svnadmin rmtxns -- 저장소(repository)로부터 트랜잭션(transaction)를 삭제합니다.

### 용법

```
svnadmin rmtxns REPOS_PATH TXN_NAME...
```

## 설명

미해결의 트랜잭션(transaction)를 저장소(repository)로부터 삭제합니다. 이것은,>그리고 자세하게 접할 수 있고 있습니다.

## 례

이름이 붙은 트랜잭션(transaction)의 삭제:

```
$ svnadmin rmtxns /usr/local/svn/repos/ 1w 1x
```

행운의 일로, svn lstxns 의 출력은, rmtxns의 입력과 같은 정도 잘 움직입니다:

```
$ svnadmin rmtxns /usr/local/svn/repos/ `svnadmin lstxns /usr/local/svn/repos/`
```

이것은 저장소(repository)로부터 모든 커밋되어 있지 않은 트랜잭션(transaction) (을)를 삭제합니다.

## svnadmin setlog

### Name

svnadmin setlog -- 리비전에 로그 메시지를 설정합니다.

## 용법

```
svnadmin setlog REPOS_PATH -r REVISION FILE
```

## 설명

FILE의 내용을 리비전 REVISION의 로그 메시지로서 설정합니다.

이것은,svn propset --revprop를 사용해 리비전상에 svn:log속성을 설정하는 것과 닮아 있습니다만, --bypass-hooks를 사용해 커밋 전후의 후의 실행을 피한다 일이 생깁니다. 이것은 pre-revprop-change hook중에서 리비전 속성의 수정 하지만 유효하지 않는 경우에 편리합니다.

| Warning                                                    |
|------------------------------------------------------------|
| 리비전 속성은 버전 관리하에는 없기 때문에 이 커멘드는 이전의 로그 메시지를 완전하게 덧쓰기해 버립니다. |

## 스윗치

```
--revision (-r) ARG
--bypass-hooks
```

## 례

파일'msg'의 내용을 리비전 19의 로그 메시지로 설정합니다:

```
$ svnadmin setlog /usr/local/svn/repos/ -r 19 msg
```

### 1.3. svnlook

**svnlook** 는 Subversion 저장소(repository)의 다른 부분을 조사하기 위한 커멘드 라인 툴입니다. 이 커멘드는 저장소(repository)에는 무슨 변경도 더하지 않습니다단지"조사하는"싶은 예 이용됩니다. **svnlook**는 저장소(repository) 혹은로 이용되는 것이 전형적입니다만, 저장소(repository) 관리자는 진단의 목적으로도 이용할 수 있는 것에 눈치챌지도 알려지지 않습니다.

**svnlook** 는 직접 저장소(repository)에 액세스 하는 형태로 실행된다 의로(그리고, 그것이 이유로 저장소(repository)의 존재하는 머신 상에서만 이용하는 것이 할 수 있습니다만), URL가 아니고, 패스 명칭에 의해 저장소(repository)를 참조합니다.

리비전이나 트랜잭션(transaction)가 지정되지 않으면, **svnlook** 는 저장소(repository)의 최신 리비전을 사용합니다.

#### 1.3.1. svnlook스윗치

**svnlook** 중의 스윗치는 **svn** 나**svnadmin**등과 함께로 글로벌하게 일합니다만, (정도)만큼 어느 스윗치는 하나의 서브 커멘드에 밖에 효과가 없습니다. 그것은**svnlook**의 기능의 유효 범위가(의도적으로) 한정되어 있기 때문입니다.

--no-diff-deleted

**svnlook**가 삭제된 파일의 차이를 표시하지 않게 합니다. 트랜잭션(transaction)/리비전중에서 파일이 삭제되었을 때의 디폴트 동작은, 그 파일이 빈 파일로서 남아 있을 때 것과 같게 됩니다.

--revision (-r)

조사하고 싶은 특정의 리비전 번호를 지정합니다.

--transaction (-t)

조사하고 싶은 특정의 트랜잭션(transaction) 번호를 지정합니다.

--show-ids

파일 시스템 트리중의, 각각의 패스마다의 파일 시스템 노드 리비전 번호를 표시합니다.

### 1.3.2. svnlook 서브 커맨드

#### Table of Contents

[svnlook author](#) -- 처리한 사람의 표시.

[svnlook cat](#) -- 파일의 내용을 표시합니다.

[svnlook changed](#) -- 변경된 패스를 표시합니다.

[svnlook date](#) -- 일자를 표시합니다.

[svnlook diff](#) -- 변경된 파일, 디렉토리의 차분을 표시합니다.

[svnlook dirs-changed](#) -- 변경이 있던 디렉토리를 표시합니다.

[svnlook help](#) --

[svnlook history](#) -- 저장소(repository)중의 패스의 히스토리에 관한 정보를 표시합니다 (혹은 패스가 지정되지 않았던 경우에는 루트 디렉토리가 됩니다).

[svnlook info](#) -- 작업자, 일자, 로그 메시지의 크기, 로그 메시지를 표시합니다.

[svnlook log](#) -- 로그 메시지를 표시합니다.

[svnlook proplist](#) -- 버전화 된 파일과 디렉토리의 속성의 이름과 값을 표시합니다.

[svnlook tree](#) -- 트리를 표시합니다.

[svnlook youngest](#) -- 제일 최근의 리비전 번호를 표시합니다.

## svnlook author

### Name

svnlook author -- 처리한 사람의 표시.

### 용법

svnlook author REPOS\_PATH

### 설명

저장소(repository)의 리비전이나 트랜잭션(transaction)를 실행했다 사람을 표시합니다.

### 스윗치

--revision (-r)

--transaction (-t)

### 예

**svnlook author**는 편리합니다만, 그만큼 재미있는 커맨드가 아닙니다:

```
$ svnlook author -r 40 /usr/local/svn/repos
sally
```

## svnlook cat

### Name

svnlook cat -- 파일의 내용을 표시합니다

### 용법

svnlook cat REPOS\_PATH PATH\_IN\_REPOS

### 설명

파일의 내용을 표시합니다.

## 스윗치

```
--revision (-r)
--transaction (-t)
```

## 예

이하는/trunk/README 에 있는 트랜잭션(transaction) ax8중의 파일의 내용을 표시하고 있습니다:

```
$ svnlook cat -t ax8 /usr/local/svn/repos /trunk/README
Subversion, a version control system.
=====
$LastChangedDate: 2003-07-17 10:45:25 -0500 (Thu, 17 Jul 2003) $
Contents:
    I.  A FEW POINTERS
    II. DOCUMENTATION
    III. PARTICIPATING IN THE SUBVERSION COMMUNITY
```

## svnlook changed

### Name

svnlook changed -- 변경된 패스를 표시합니다.

### 용법

```
svnlook changed REPOS_PATH
```

### 설명

"svn update-style" 의 체일 스테이터스 캐릭터와 같이 특정의 리비전이나 트랜잭션(transaction)로 변경된 패스를 표시합니다: A 는 추가, D 는 삭제, U 는 수정을 나타냅니다.

## 스윗치

```
--revision (-r)
--transaction (-t)
```

## 예

테스트 저장소(repository)의 리비전 99로 수정한 모든 파일의 일람을 표시합니다:

```
$ svnlook changed -r 39 /usr/local/svn/repos
A   trunk/vendors/deli/
A   trunk/vendors/deli/chips.txt
A   trunk/vendors/deli/sandwich.txt
A   trunk/vendors/deli/pickle.txt
```

## svnlook date

### Name

svnlook date -- 일자를 표시합니다.

### 용법

```
svnlook date REPOS_PATH
```

### 설명

저장소(repository)중의 특정 리비전이나 트랜잭션(transaction)의 일자를 표시합니다.

## 스윗치

```
--revision (-r)
--transaction (-t)
```

## 예

이것은, 테스트 저장소(repository)의 리비전 40의 일자를 표시합니다:

```
$ svnlook date -r 40 /tmp/repos/
2003-02-22 17:44:49 -0600 (Sat, 22 Feb 2003)
```

## svnlook diff

### Name

svnlook diff -- 변경된 파일, 디렉토리의 차분을 표시합니다.

### 용법

```
svnlook diff REPOS_PATH
```

### 설명

저장소(repository)중에서 변경된 파일, 속성의 차분을 GNU 형식에서 표시합니다.

## 스윗치

```
--revision (-r)
--transaction (-t)
--no-diff-deleted
```

## 예

이것은, 새롭게 추가된 파일(빈 파일), 삭제된 파일, 카피되었다 파일을 표시합니다:

```
$ svnlook diff -r 40 /usr/local/svn/repos/
Copied: egg.txt (from rev 39, trunk/vendors/deli/pickle.txt)

Added: trunk/vendors/deli/soda.txt
=====

Modified: trunk/vendors/deli/sandwich.txt
=====
--- trunk/vendors/deli/sandwich.txt      (original)
+++ trunk/vendors/deli/sandwich.txt      2003-02-22 17:45:04. 000000000 -0600
@@ -0, 0 +1 @@
+Don't forget the mayo!

Deleted: trunk/vendors/deli/chips.txt
=====

Deleted: trunk/vendors/deli/pickle.txt
=====
```

## svnlook dirs-changed

### Name

svnlook dirs-changed -- 변경이 있던 디렉토리를 표시합니다.

### 용법

```
svnlook dirs-changed REPOS_PATH
```

### 설명

(속성을 편집에 의해) 그것 자신으로 변경이 있었는지, 그 아이의 파일로 변경이 있던 디렉토리를 표시합니다.

## 스윗치

```
--revision (-r)
--transaction (-t)
```

## 예

우리의 샘플 저장소(repository)중의 리비전 40으로 수정되었다 디렉토리를 표시합니다:

```
$ svnlook dirs-changed -r 40 /usr/local/svn/repos
trunk/vendors/deli/
```

# svnlook help

## Name

svnlook help

## 용법

Also svnlook -h and svnlook -?.

## 설명

svnlook 의 헬프를 표시합니다. **svn help**의 헬프와 같아, 무엇인가가 모르게 되었을 때에는 항상 도움이 됩니다.

# svnlook history

## Name

svnlook history -- 저장소(repository)중의 패스의 히스토리에 관한 정보를 표시합니다 (혹은 패스가 지정되지 않았던 경우에는 루트 디렉토리가 됩니다).

## 용법

```
svnlook history REPOS_PATH
[PATH_IN_REPOS]
```

## 설명

저장소(repository)중의 패스의 히스토리에 관한 정보를 표시합니다 (혹은 패스가 지정되지 않았던 경우에는 루트 디렉토리가 됩니다).

## 스윗치

```
--revision (-r)
--show-ids
```

## 예

이하는 예로서 만든 저장소(repository)의 리비전 15 로 보았다 패스/tags/1. 0의 히스토리 표시입니다.

```
$ svnlook history -r 20 /usr/local/svn/repos /tags/1. 0 --show-ids
```

| REVISION | PATH ID                    |
|----------|----------------------------|
| 19       | /tags/1. 0 1. 2.12         |
| 17       | /branches/1. 0-rc2 1. 1.10 |
| 16       | /branches/1. 0-rc2 1. 1. x |
| 14       | /trunk 1. 0. q             |
| 13       | /trunk 1. 0. o             |
| 11       | /trunk 1. 0. k             |
| 9        | /trunk 1. 0. g             |
| 8        | /trunk 1. 0. e             |
| 7        | /trunk 1. 0. b             |
| 6        | /trunk 1. 0.9              |
| 5        | /trunk 1. 0.7              |
| 4        | /trunk 1. 0.6              |
| 2        | /trunk 1. 0.3              |



1 /trunk 1. 0.2

## svnlook info

### Name

svnlook info -- 작업자, 일자, 로그 메시지의 크기, 로그 메시지를 표시합니다.

### 용법

```
svnlook info REPOS_PATH
```

### 설명

작업자, 일자, 로그 메시지의 크기, 로그 메시지를 표시합니다.

### 스위치

```
--revision (-r)
--transaction (-t)
```

### 예

이것은 샘플 저장소(repository)의 리비전 40에 대한 info 의 출력입니다.

```
$ svnlook info -r 40 /usr/local/svn/repos
sally
2003-02-22 17:44:49 -0600 (Sat, 22 Feb 2003)
15
Rearrange lunch.
```

## svnlook log

### Name

svnlook log -- 로그 메시지를 표시합니다.

### 용법

```
svnlook log REPOS_PATH
```

### 설명

로그 메시지를 표시합니다.

### 스위치

```
--revision (-r)
--transaction (-t)
```

### 예

이것은 샘플 저장소(repository)의 리비전 40의 로그 표시입니다:

```
$ svnlook log /tmp/repos/
Rearrange lunch.
```

## svnlook proplist

### Name

svnlook proplist -- 버전화 된 파일과 디렉토리의 속성의 이름과 값을 표시합니다.

## 용법

```
svnlook proplist REPOS_PATH PATH_IN_REPOS
```

## 설명

저장소(repository)중의 패스의 속성을 일람표 가리킵니다. -v를 사용해 속성치도 표시할 수 있습니다.

## 스윗치

```
--revision (-r)
--transaction (-t)
--verbose (-v)
```

## 예

이것은 HEAD 리비전에 있는 파일/trunk/README (으)로 설정된 속성명을 표시하고 있습니다:

```
$ svnlook proplist /usr/local/svn/repos /trunk/README
original-author
svn:mime-type
```

이것은 전의 예와 같은 커멘드입니다만, 이번은 속성치도 함께 표시 하고 있습니다:

```
$ svnlook proplist /usr/local/svn/repos /trunk/README
original-author : fitz
svn:mime-type : text/plain
```

# svnlook tree

## Name

svnlook tree -- 트리를 표시합니다.

## 용법

```
svnlook tree REPOS_PATH [PATH_IN_REPOS]
```

## 설명

PATH\_IN\_REPOS 로부터 시작되는 트리를 표시합니다. (PATH\_IN\_REPOS 의 지정이 없는 경우에는 루트로부터 시작되는 트리를 표시합니다. ). 옵션으로 노드 리비전 id를 표시시킬 수도 있습니다.

## 스윗치

```
--revision (-r)
--transaction (-t)
--show-ids
```

## 예

이것은, (노드 번호 돌출하고) 샘플 저장소(repository)의 리비전 40의 트리를 표시한 것입니다:

```
$ svnlook tree -r 40 /usr/local/svn/repos --show-ids
/ 0. 0.2j
  trunk/ p. 0.2j
    vendors/ q. 0.2j
      deli/ 1g. 0.2j
        egg.txt 1i.e. 2j
        soda.txt 1k. 0.2j
        sandwich.txt 1j. 0.2j
```

# svnlook youngest

## Name

svnlook youngest -- 제일 최근의 리비전 번호를 표시합니다.

## 용법

```
svnlook youngest REPOS_PATH
```

## 설명

저장소(repository)에 있는 제일 최근의 리비전 번호를 표시합니다.

## 예

이것은, 샘플 저장소(repository)의 제일 최근의 리비전 번호를 표시하고 있습니다:

```
$ svnlook youngest /tmp/repos/  
42
```

## Notes

[\[1\]](#) 네, 그렇네요. --version 스위치를 사용할 때에는 서브 커멘드는 불필요합니다. 조금 다음에 그것에 접합니다.

---

[EditText](#) | [Refresh](#) | [FindPage](#) | [DeletePage](#) | [LikePages](#) | [Tour](#) | [Keywords](#) |

  

Last modified: 2005-06-29 18:14:36, Processing time: 0.0995 sec

[Corporate File Exchange](#)

Secure, Large File Exchange in Federated Scheduled and Real Time, Unix and Environments.

[RepliWeb File Replication](#)

Windows, Files Synchronization